



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

Consultas sobre las funciones Asincronas y Sincronicas

Presentado a: Instructora Yoly Alexandra

Por Aprendiz: Cristian David Yalanda

Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **14** de **09** del año **2026**



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

Tabla de Contenido

Contenido

Actividad de Aprendizaje	3
Enunciado – 1 Objetivo	3
Solución	3
Enunciado – 2 Investigacion.....	4
Solución	4
Enunciado – 3 Tabla comparativa	6
Solución	6
Enunciado – 4 Comprendiendo el funcionamiento	7
Solución	7
Enunciado – 5 Analizando una función asíncrona	8
Solución	9
Enunciado – 6 Prediga el resultado.....	9
Solución	10
Enunciado – 7 Funciones con async y await	11
Solución	11
Enunciado – 8 ¿Cuándo utilizar cada una?	12
Solución	13
Enunciado – 9 Situación problema	13
Solución	14
Enunciado – 10 Reto práctico	15
Solución	15
Enunciado – 11 Reflexión final	17
Solución	17



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Actividad de Aprendizaje

Funciones Síncronas y Asíncronas en JavaScript

Enunciado – 1 Objetivo

Comprender la diferencia entre las funciones síncronas y asíncronas en JavaScript, identificando su funcionamiento, características, ventajas, casos de uso y situaciones en las que es conveniente utilizar cada una.

Solución

Diferencia principal

síncronas	asíncronas
ejecutan las tareas en orden	permiten continuar mientras una tarea espera
bloquean el flujo hasta terminar	no bloquean el flujo principal
son más sencillas de entender	requieren comprender callbacks, promesas o <code>async/await</code>
útiles para tareas rápidas	útiles para tareas que pueden tardar
ejemplo: operaciones matemáticas	ejemplo: <code>setTimeout()</code> o consultas a datos

Ventajas

síncronas:

- son fáciles de escribir y comprender;
- el código se ejecuta de manera predecible;
- son adecuadas para operaciones rápidas.

asíncronas:

- permiten que la página siga respondiendo;
- son útiles cuando una tarea tarda en completarse;
- permiten trabajar con operaciones como peticiones a servidores y temporizadores

¿cuándo utilizar cada una?

las funciones síncronas son convenientes cuando la operación es rápida y no necesitamos esperar ningún proceso externo. las funciones asíncronas son convenientes cuando tenemos que esperar una respuesta o proceso que puede tardar, por ejemplo, consultar datos de un servidor, cargar información o utilizar un temporizador



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Enunciado – 2 Investigacion

Investigue los siguientes conceptos:

- ¿Qué es una función síncrona?
- ¿Qué es una función asíncrona?
- ¿Cómo se ejecuta una función síncrona?
- ¿Cómo se ejecuta una función asíncrona?
- ¿Qué significa que una operación sea bloqueante?
- ¿Qué significa que una operación sea no bloqueante?
- ¿Qué es una Promise?
- ¿Para qué sirven async y await?
- ¿Qué problemas pueden solucionarse utilizando programación asíncrona?

Solución

1. ¿qué es una función síncrona?

una función síncrona es aquella que ejecuta sus instrucciones una después de otra y en orden. el programa debe esperar a que termine una operación antes de continuar con la siguiente

2. ¿qué es una función asíncrona?

una función asíncrona permite realizar operaciones que pueden tardar sin detener completamente la ejecución del programa. mientras una operación está esperando, javascript puede continuar realizando otras tareas. Algunos ejemplos son las peticiones a servidores, temporizadores y operaciones que necesitan esperar una respuesta

3. ¿cómo se ejecuta una función síncrona?

se ejecuta de manera secuencial, siguiendo el orden en que aparecen las instrucciones.

por ejemplo:

```
console.log("inicio");  
console.log("proceso");  
console.log("fin");
```

el resultado será:



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

inicio

proceso

fin

cada instrucción termina antes de que comience la siguiente

4. ¿cómo se ejecuta una función asíncrona?

Una función asíncrona puede iniciar una operación y permitir que javascript continúe ejecutando otras instrucciones mientras espera el resultado

5. ¿qué significa que una operación sea bloqueante?

una operación bloqueante es aquella que detiene la ejecución del programa hasta que termina. Por ejemplo, si una tarea tarda mucho tiempo, las instrucciones que vienen después tendrán que esperar. En pocas palabras: Una operación bloqueante hace esperar al programa

6. ¿qué significa que una operación sea no bloqueante?

una operación no bloqueante permite continuar sin esperar a que termine la tarea. Esto es muy útil en aplicaciones web porque evita que la página quede detenida mientras espera una respuesta

7. ¿qué es una promise?

una promise (promesa) es un objeto de javascript que representa el resultado futuro de una operación asíncrona. Una promise puede encontrarse principalmente en tres estados:

- pending: la operación todavía está pendiente;
- fulfilled: la operación terminó correctamente;
- rejected: la operación terminó con un error.

8. ¿para qué sirven async y await?

async y await sirven para trabajar con operaciones asíncronas de una manera más sencilla y fácil de leer.

async se utiliza para indicar que una función es asíncrona

await permite esperar el resultado de una promise dentro de una función async

9. ¿qué problemas pueden solucionarse utilizando programación asíncrona?

las funciones síncronas ejecutan las instrucciones una por una y esperan a que cada tarea termine. las funciones asíncronas, en cambio, permiten continuar con otras tareas mientras se espera una operación que puede tardar. Las promises, junto con async y



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL

FORMATO ENTREGA DE EVIDENCIAS

await, facilitan el manejo de estas operaciones asíncronas y permiten crear aplicaciones web más rápidas y fluidas

Enunciado – 3 Tabla comparativa

Complete la siguiente tabla:

Solución

característica	funciones síncronas	funciones asíncronas
¿cómo se ejecutan?	se ejecutan una instrucción después de otra	pueden iniciar una operación y continuar con otras instrucciones mientras esperan
orden de ejecución	siguen el orden establecido en el código	no necesariamente terminan en el mismo orden en que comenzaron
¿bloquean la ejecución?	sí, una tarea puede detener el flujo hasta terminar	normalmente no bloquean el flujo principal mientras esperan
¿esperan una operación?	sí, esperan a que termine antes de continuar	pueden esperar el resultado sin detener las demás tareas
tiempo de respuesta	puede aumentar si una operación tarda mucho	puede ofrecer una mejor respuesta cuando hay operaciones que tardan
uso de promise	normalmente no necesitan promises	las promises se utilizan frecuentemente para manejar operaciones asíncronas
uso de async/await	no es necesario	se utiliza para trabajar de forma más sencilla con promises
ejemplos	operaciones matemáticas, asignación de variables y funciones normales	setTimeout(), peticiones a servidores y consultas a una api
¿cuándo se utilizan?	cuando las operaciones son rápidas y no necesitan esperar respuestas externas	cuando una operación puede tardar, como cargar datos o comunicarse con un servidor
ventajas	son sencillas de entender y tienen un orden de ejecución claro	permiten realizar otras tareas mientras se espera y evitan bloquear la aplicación



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

característica	funciones síncronas	funciones asíncronas
desventajas	una operación lenta puede detener las instrucciones siguientes	pueden ser más difíciles de entender y requieren manejar promises, errores y tiempos de espera

Enunciado – 4 Comprendiendo el funcionamiento

Analice el siguiente código:

```
console.log("Inicio");
```

```
function saludar() {  
    console.log("Hola aprendiz");  
}
```

```
saludar();
```

```
console.log("Fin");
```

Responda:

1. ¿Qué mensaje aparece primero?
2. ¿Qué mensaje aparece segundo?
3. ¿Qué mensaje aparece tercero?
4. ¿La función saludar() debe terminar para continuar con la siguiente instrucción?
5. ¿Este código es síncrono o asíncrono?
6. Explique por qué.

Solución

1. ¿qué mensaje aparece primero?

aparece primero:

"inicio"

2. ¿qué mensaje aparece segundo?

aparece segundo:

"hola aprendiz"



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

3. ¿qué mensaje aparece tercero?

aparece tercero:

"fin"

4. ¿la función saludar() debe terminar para continuar con la siguiente instrucción?

sí. javascript debe ejecutar y terminar la función saludar() antes de continuar con `console.log("fin");`

5. ¿este código es síncrono o asíncrono?

es **síncrono**.

6. explique por qué.

Es síncrono porque las instrucciones se ejecutan una por una y en el orden en que aparecen. primero se muestra "inicio", después se ejecuta la función saludar () y se muestra "hola aprendiz", y finalmente se muestra "fin". la siguiente instrucción espera a que termine la anterior

Enunciado – 5 Analizando una función asíncrona

Observe:

```
console.log("Inicio");
```

```
setTimeout(() => {  
    console.log("Proceso terminado");  
}, 2000);
```

```
console.log("Fin");
```

Responda:

1. ¿Qué mensaje aparece primero?
2. ¿Qué mensaje aparece inmediatamente después?
3. ¿Qué mensaje aparece aproximadamente después de 2 segundos?
4. ¿Por qué "Fin" aparece antes de "Proceso terminado"?
5. ¿El programa se detiene durante los 2 segundos?
6. ¿Qué característica de la programación asíncrona se puede observar?



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Solución

1. ¿qué mensaje aparece primero?

Aparece primero:

"inicio"

2. ¿qué mensaje aparece inmediatamente después?

Aparece inmediatamente después:

"fin"

3. ¿qué mensaje aparece aproximadamente después de 2 segundos?

Aparece:

"proceso terminado"

4. ¿por qué "fin" aparece antes de "proceso terminado"?

Porque `setTimeout()` es una operación asíncrona. javascript programa la función para que se ejecute después de aproximadamente 2 segundos y continúa con la siguiente instrucción, que es `console.log("fin");`

5. ¿el programa se detiene durante los 2 segundos?

No. el programa continúa ejecutando otras instrucciones mientras espera que termine el tiempo establecido en `setTimeout()`

6. ¿qué característica de la programación asíncrona se puede observar?

Se observa que una operación no bloquea la ejecución del programa. mientras `setTimeout()` espera los 2 segundos, javascript continúa ejecutando las siguientes instrucciones.

Enunciado – 6 Prediga el resultado

Prediga el resultado

Sin ejecutar el código, indique cuál será el orden de salida:

```
console.log("A");
```

```
setTimeout(() => {  
  console.log("B");  
}, 0);
```

```
console.log("C");
```

```
setTimeout(() => {
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
    console.log("D");  
  }, 1000);
```

```
console.log("E");
```

Orden esperado:

Explique:

¿Por qué B no aparece inmediatamente después de A, aunque el tiempo establecido en `setTimeout()` sea 0?

Solución

`setTimeout()` es una función asíncrona. Aunque el tiempo establecido sea 0 milisegundos, la función no se ejecuta inmediatamente.

Primero javascript ejecuta todas las instrucciones síncronas:

1. aparece a;
2. se programa b con `setTimeout()`;
3. aparece c;
4. se programa d para después de 1 segundo;
5. aparece e

Cuando termina de ejecutar las instrucciones principales, javascript puede ejecutar la función que estaba esperando, por eso aparece b después de e.

finalmente, después de aproximadamente 1 segundo, aparece d.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Enunciado – 7 Funciones con async y await

Analice:

```
function obtenerUsuario() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve("Usuario encontrado");  
    }, 2000);  
  });  
}
```

```
async function mostrarUsuario() {  
  
  console.log("Buscando usuario...");  
  
  const resultado = await obtenerUsuario();  
  
  console.log(resultado);  
  
  console.log("Proceso terminado");  
}
```

mostrarUsuario();

Responda:

1. ¿Qué devuelve obtenerUsuario()?
2. ¿Qué representa resolve()?
3. ¿Qué función está definida como async?
4. ¿Qué hace await?
5. ¿Qué ocurre mientras se esperan los 2 segundos?
6. ¿Por qué async y await facilitan la lectura del código asíncrono?
7. ¿Qué ocurriría si se eliminara await?

Solución

1. ¿qué devuelve obtenerusuario()?

devuelve una promise. esta promise se resuelve después de aproximadamente 2 segundos con el mensaje "usuario encontrado"



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

2. ¿qué representa resolve()?

resolve() indica que la promise terminó correctamente y entrega el resultado de la operación

3. ¿qué función está definida como async?

la función definida como async es:

```
async function mostrarusuario()
```

4. ¿qué hace await?

await espera a que la promise termine para poder obtener su resultado.

en este caso:

```
const resultado = await obtenerusuario();
```

espera aproximadamente 2 segundos y después guarda "usuario encontrado" en resultado

5. ¿qué ocurre mientras se esperan los 2 segundos?

la función mostrarusuario() queda esperando el resultado de la promise, pero javascript no bloquea completamente el programa. puede continuar realizando otras tareas mientras espera

6. ¿por qué async y await facilitan la lectura del código asíncrono?

porque permiten escribir código asíncrono de una forma más sencilla y ordenada, parecida a la forma en que se lee un código síncrono. En lugar de utilizar varios métodos para manejar una promise, podemos utilizar await para esperar directamente su resultado

7. ¿qué ocurriría si se eliminara await?

si se elimina await, resultado recibiría la promise en lugar del texto "usuario encontrado".

por ejemplo:

```
const resultado = obtenerusuario();
```

```
console.log(resultado);
```

Se mostraría la promise, porque todavía no habría terminado la operación. Por eso await es necesario en este caso para esperar el resultado antes de mostrarlo.

Enunciado – 8 ¿Cuándo utilizar cada una?

Complete la siguiente actividad.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Indique si utilizaría una función SÍNCRONA (S) o ASÍNCRONA (A).

Solución

situación	s / a	¿por qué?
sumar dos números	s	es una operación rápida que no necesita esperar una respuesta externa.
validar que un campo no esté vacío	s	la validación se realiza inmediatamente con los datos disponibles.
consultar información de una api	a	se debe esperar una respuesta de un servidor.
consultar una base de datos	a	la consulta puede tardar y requiere esperar una respuesta.
esperar una respuesta de un servidor	a	la respuesta puede tardar y no es necesario bloquear el programa mientras llega.
calcular el total de una factura	s	es un cálculo que puede realizarse inmediatamente con los datos disponibles.
leer información de un archivo de manera asíncrona	a	la lectura puede tardar y se realiza sin bloquear la ejecución principal.
realizar una animación con retraso	a	se necesita esperar un tiempo antes de ejecutar una acción.
procesar una operación que depende de una api	a	primero se debe esperar la respuesta de la api para continuar.
mostrar el resultado de una operación matemática	s	el resultado ya está disponible y puede mostrarse inmediatamente.

Enunciado – 9 Situación problema

Una empresa necesita desarrollar una aplicación web para consultar información de sus clientes.

Cuando el usuario presiona el botón:

“Consultar cliente”

la aplicación debe:

- Enviar una solicitud al servidor.
- Esperar la respuesta.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

- Recibir los datos del cliente.
- Mostrar la información en pantalla.

Preguntas

- ¿Utilizaría una función síncrona o asíncrona?
- ¿Por qué?
- ¿Qué problema podría presentarse si la aplicación bloqueara completamente la interfaz mientras espera la respuesta?
- ¿Qué tecnologías de JavaScript podrían utilizarse?
 - Promise
 - async
 - await
 - fetch

Explique la función de cada una.

Solución

- ¿utilizaría una función síncrona o asíncrona?

utilizaría una función asíncrona

- ¿por qué?

Porque la aplicación necesita enviar una solicitud al servidor y esperar una respuesta. esta operación puede tardar, por lo que es mejor realizarla de forma asíncrona para que la aplicación pueda continuar funcionando mientras llega la información

- ¿qué problema podría presentarse si la aplicación bloqueara completamente la interfaz mientras espera la respuesta?

La interfaz podría quedar congelada o sin responder mientras el servidor procesa la solicitud. el usuario no podría interactuar normalmente con la página hasta recibir la respuesta. Esto generaría una mala experiencia para el usuario

- ¿qué tecnologías de javascript podrían utilizarse?

tecnología	función
promise	representa el resultado futuro de una operación asíncrona, como una solicitud al servidor. puede indicar si la operación terminó correctamente o tuvo un error.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

async	se utiliza para indicar que una función trabajará con operaciones asíncronas y promises.
await	permite esperar el resultado de una promise dentro de una función async antes de continuar con la siguiente instrucción.
fetch	permite realizar solicitudes a un servidor o api para enviar o recibir información. devuelve una promise.

Enunciado – 10 Reto práctico

Construya una función que simule la consulta de un usuario:

```
function consultarUsuario() {  
  
    // Simular consulta al servidor  
  
}
```

La función debe tardar 3 segundos en responder.

Después debe mostrar:

Consultando usuario...
Usuario encontrado
Proceso terminado

Requisito

Utilice:

- Promise
- setTimeout()
- async
- await

Solución

HTML

```
<!DOCTYPE html>  
<html lang="en">  
<head>
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>consulta de usuario</title>
</head>
<body>

  <h1>consulta de usuario</h1>

  <button id="boton">consultar usuario</button>

  <p id="resultado"></p>

  <script src="index.js"></script>
</body>
</html>
```

JS

```
function consultarusuario() {

  return new Promise(function(resolve) {

    setTimeout(function() {

      resolve("usuario encontrado");

    }, 3000);

  });

}

async function mostrarusuario() {

  document.getElementById("resultado").innerHTML = "consultando usuario...";

  let resultado = await consultarusuario();

  document.getElementById("resultado").innerHTML = resultado;

}
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
document.getElementById("resultado").innerHTML += "<br>proceso  
terminado";  
}  
  
document.getElementById("boton").onclick = mostrarusuario;
```

consulta de usuario

consultar usuario

Antes de hacer la consulta

consulta de usuario

consultar usuario

usuario encontrado
proceso terminado

Después de hacer la consulta

Enunciado – 11 Reflexión final

Responda con sus propias palabras:

1. ¿Cuál es la principal diferencia entre una función síncrona y una asíncrona?
2. ¿Por qué las operaciones que dependen de un servidor normalmente se manejan de forma asíncrona?
3. ¿Qué ventaja ofrece async/await frente a trabajar directamente con múltiples then()?
4. ¿Una función asíncrona significa necesariamente que se ejecuta en otro hilo? Explique.
5. Mencione tres situaciones reales en las que utilizaría programación asíncrona.

Solución

1. ¿cuál es la principal diferencia entre una función síncrona y una asíncrona?

La principal diferencia es que una función síncrona ejecuta las instrucciones una por una y espera a que cada operación termine para continuar. una función asíncrona puede esperar una operación sin bloquear la ejecución del programa

2. ¿por qué las operaciones que dependen de un servidor normalmente se manejan de forma asíncrona?



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Porque la respuesta del servidor puede tardar. si se hiciera de forma síncrona, la aplicación podría quedarse esperando y la interfaz podría dejar de responder. de forma asíncrona, el programa puede continuar funcionando mientras llega la respuesta

3. ¿qué ventaja ofrece `async/await` frente a trabajar directamente con múltiples `then()`?

`async/await` hace que el código sea más fácil de leer y entender, porque permite escribir las operaciones asíncronas de una manera más parecida al código normal. también facilita seguir el orden de las operaciones

4. ¿una función asíncrona significa necesariamente que se ejecuta en otro hilo? explique.

No. que una función sea asíncrona no significa necesariamente que se ejecute en otro hilo. javascript puede manejar operaciones asíncronas mientras continúa ejecutando otras tareas. la programación asíncrona se relaciona principalmente con no bloquear la ejecución, no necesariamente con crear otro hilo

5. mencione tres situaciones reales en las que utilizaría programación asíncrona.

- consultar información de una api, por ejemplo, obtener los datos de un cliente
- enviar información a un servidor, como registrar un usuario en una página web
- consultar una base de datos, mientras la aplicación continúa funcionando mientras espera la respuesta